

ReAct (Reason + Action) Loop

1. Introduction

An LLM can generate responses, but real-world applications often require it to interact with external systems.

Consider an order processing system.

A user asks:

"Check order ORD1001 and cancel it if it has not been delivered."

The LLM does not directly know the order status. It needs to use a tool to retrieve that information and then decide what to do.

This is where the **ReAct (Reason + Action)** pattern is useful.

The basic cycle is:

```
Reason
  ↓
Action
  ↓
Observation
  ↓
Reason
  ↓
Action
  ↓
...
  ↓
Final Answer
```

2. The Order Processing Example

Assume the application provides two tools:

```
get_order()
cancel_order()
```

The request is:

Check order ORD1001 and cancel it if it has not been delivered.

The LLM initially does not know the order status.

It therefore decides:

```
Reason:  
I need to check the order status.  
  
Action:  
get_order("ORD1001")
```

The application executes the tool and gets:

```
Observation:  
{  
  "status": "SHIPPED",  
  "customer": "John",  
  "amount": 2500  
}
```

The observation is then provided back to the LLM.

The LLM reasons again:

```
Reason:  
The order is shipped but not delivered.  
The user asked me to cancel it if it has not been delivered.  
  
Action:  
cancel_order("ORD1001")
```

The tool returns:

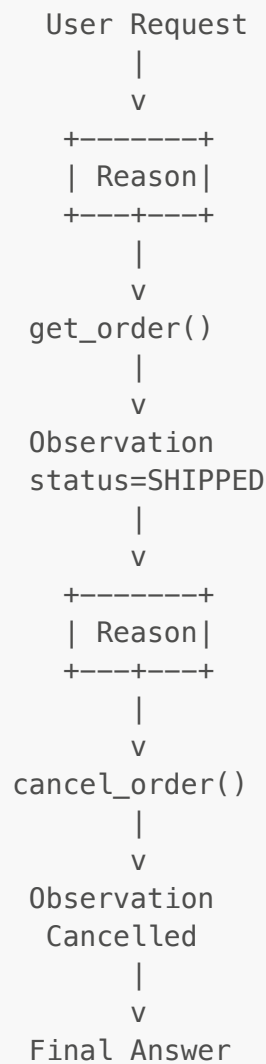
```
Observation:  
Order ORD1001 has been cancelled.
```

The LLM can now provide the final answer:

```
Order ORD1001 has been cancelled.
```

3. The Complete ReAct Cycle

The entire interaction can be represented as:



The important point is that **the second action depends on the result of the first action.**

4. Implementing the Concept with LangChain

Tools can be defined using LangChain's `@tool` decorator:

```

from langchain_core.tools import tool

@tool
def get_order(order_id: str):
    """Get order details."""

    orders = {
        "ORD1001": {
            "status": "SHIPPED",
            "customer": "John"
        },
        "ORD1002": {
            "status": "DELIVERED",

```

```
        "customer": "Alice"
    }
}

return orders.get(order_id, "Order not found")

@tool
def cancel_order(order_id: str):
    """Cancel an order."""

    return f"Order {order_id} has been cancelled."
```

The tools can be made available to the LLM:

```
tools = [get_order, cancel_order]

llm_with_tools = llm.bind_tools(tools)
```

The LLM can now request a tool when it needs additional information or needs to perform an operation.

5. The ReAct Loop in Python

A simplified version of the execution loop is:

```
while True:

    # Reason
    response = llm_with_tools.invoke(messages)

    messages.append(response)

    # No tool call means the LLM has finished
    if not response.tool_calls:
        print(response.content)
        break

    # Action
    for tool_call in response.tool_calls:

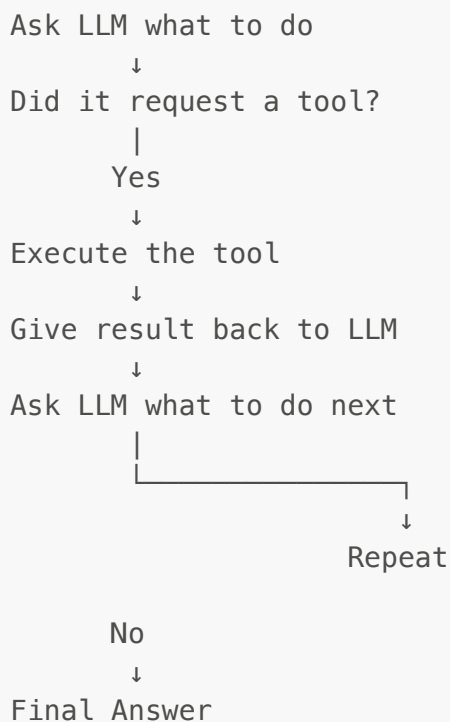
        tool_name = tool_call["name"]
        tool_args = tool_call["args"]

        selected_tool = next(
            tool for tool in tools
            if tool.name == tool_name
        )
```

```
result = selected_tool.invoke(tool_args)

# Observation
messages.append({
    "role": "tool",
    "tool_call_id": tool_call["id"],
    "content": str(result)
})
```

The essential logic is:



6. ReAct vs. Simple Tool Calling

Tool calling and ReAct are related but are not exactly the same.

A simple tool-calling interaction might be:

```
User
↓
LLM
↓
get_order()
↓
Result
```

A ReAct-style interaction can involve multiple dependent actions:

```
User
↓
Reason
↓
get_order()
↓
Observation
↓
Reason
↓
cancel_order()
↓
Observation
↓
Final Answer
```

The key characteristic of ReAct is the **iterative loop**.

7. Key Takeaway

ReAct allows an LLM to solve a task incrementally instead of trying to produce the answer in a single step.

The core pattern is:

```
Reason
↓
Action
↓
Observation
↓
Reason again
↓
Action
↓
Observation
↓
Final Answer
```

In the order-processing example:

```
Check order
↓
Get order status
↓
Observe SHIPPED
↓
Decide cancellation is required
```

↓
Cancel order
↓
Observe successful cancellation
↓
Respond to user

The central idea is simple:

The result of one action becomes information for the next reasoning step.